

딥러닝 파이토치 교과서 8장

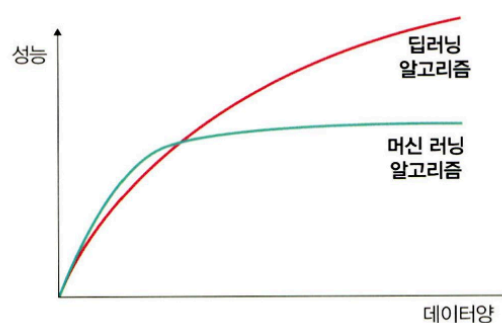
📄 제출	제출 전
📄 과제 유형	예습
📅 마감일	@2025년 6월 23일
☑ 발표	☐
☑ 완료	☐
📄 주차	14주차

8.1 성능 최적화

8.1.1 데이터를 사용한 성능 최적화

- 데이터를 사용한 성능 최적화 방법은 많은 데이터를 수집하는 것이지만, 데이터 수집이 여의치 않은 경우 임의로 데이터를 생성하는 방법도 존재
 - **최대한 많은 데이터 수집하기:** 일반적으로 딥러닝이나 머신 러닝 알고리즘은 데이터 양이 많을 수록 성능이 좋다.

▼ 그림 8-1 데이터와 딥러닝, 머신 러닝 알고리즘의 성능 비교



- **데이터 생성하기** (데이터 augmentation)
- **데이터 범위(scale) 조정하기:** 활성화 함수로 시그모이드를 사용한다면 데이터셋 범위를 0~1의 값을 갖도록 하고, 하이퍼볼릭 탄젠트를 사용한다면 데이터셋 범위를 -1~1의 값을 갖도록 조정할 수 있음

- 정규화, 규제화, 표준화도 성능 향상에 도움이 된다.

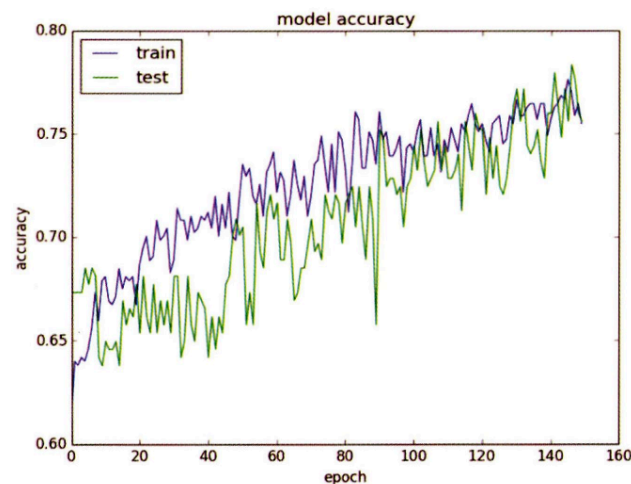
8.1.2 알고리즘을 이용한 성능 최적화

- 머신 러닝과 딥러닝을 위한 알고리즘 중 모델을 선택하여 훈련시켜 보고 최적의 성능을 보이는 알고리즘을 선택해야한다. 머신 러닝에서는 데이터 분류를 위해 SVM, KNN 알고리즘들을 선택해 훈련 시켜 보거나, 시계열 데이터의 경우 RNN, LSTM, GRU 등의 알고리즘을 훈련시켜 성능이 가장 좋은 모델을 선택하여 사용한다.

8.1.3 알고리즘 튜닝을 위한 성능 최적화

- 성능 최적화를 하는데 가장 많은 시간이 소요되는 부분이다. 모델을 하나 선택하여 훈련 시키려면 다양한 하이퍼 파라미터를 변경하면서 훈련시키고 최적의 성능을 도출해야한다. 이때 선택할 수 있는 하이퍼파라미터는 다음과 같다.
 - **진단:** 성능 향상이 어느 순간 멈췄다면 원인을 분석할 필요가 있다. 문제를 진단하는데 사용할 수 있는 것이 모델에 대한 평가이다. 다음과 같은 평가 결과를 바탕으로 모델이 과적합인지 혹은 다른 원인으로 성능 향상에 문제가 있는지에 대한 인사이트를 얻을 수 있다.

▼ 그림 8-2 알고리즘 성능 진단



- 훈련 성능이 검증(test)보다 눈에 띄게 좋다면 과적합을 의심해 볼 수 있으며, 이것을 해결하기 위해 규제화를 진행한다면 성능 향상에 도움이 된다.
- 훈련과 검증 결과가 모두 성능이 좋지 않다면 과소 적합을 의심할 수 있다. 과소 적합 상황에서는 네트워크 구조를 변경하거나 훈련을 늘리기 위해 에포크 수를 조정해볼 수 있다.
- 훈련 성능이 검증을 넘어서는 변곡점이 있다면 조기 종료를 고려할 수 있다.

- **가중치:** 가중치에 대한 초깃값은 작은 난수를 사용한다. 작은 난수의 숫자가 애매하다면 오토 인코더 같은 비지도 학습을 이용해 사전 훈련을 진행한 후 지도 학습을 진행하는 것도 방법이다.
- **학습률:** 학습률은 모델의 네트워크 구성에 따라 다르기 때문에 초기에 매우 크거나 작은 임의의 난수를 선택하여 학습 결과를 보고 조금씩 변경해야 한다. 이때 네트워크의 계층이 많다면 학습률이 높아야 하며, 네트워크의 계층이 몇 개 되지 않는다면 학습률은 작게 설정해야 한다.
- **활성화 함수:** 활성화 함수의 변경은 손실 함수도 함께 변경하는 경우가 많기 때문에 신중하게 해야 한다. 다루고자 하는 데이터 유형 및 데이터로 어떤 결과를 얻고 싶은지 정확하게 이해하지 못했다면 활성화 함수의 변경은 신중해야 한다. 일반적으로 활성화 함수로 sigmoid나 tanh를 사용했다면 출력 층에서는 softmax나 sigmoid 함수를 많이 선택함.
- **배치와 에포크:** 일반적으로 큰 에포크와 작은 배치를 사용하는 것이 최근 딥러닝의 트렌드이긴 하지만, 적절한 배치 크기를 위해 훈련 데이터셋의 크기와 동일하게 하거나 하나의 배치로 훈련시켜 보는 등 다양한 테스트를 진행하는 것이 좋다.
- **옵티마이저 및 손실 함수:** 일반적으로 옵티마이저는 확률적 경사 하강법을 많이 사용한다. 네트워크 구성에 따라 차이는 있지만 아담(Adam)이나 RMSProp 등이 좋은 성능을 보인다. 하지만 역시 다양한 옵티마이저와 손실 함수를 적용해보고 성능이 좋은 것을 선택해야 한다.
- **네트워크 구성:** 네트워크 구성은 네트워크 토폴로지라고도 한다. 최적의 네트워크를 구성하는 것 역시 쉽게 알 수 있는 부분이 아니기 때문에 네트워크 구성을 변경해가면서 성능을 테스트해야 한다. 예를 들어 하나의 은닉층에 뉴런을 여러 개 포함시키거나 (넓은 네트워크), 네트워크 계층을 늘리되 뉴런 개수는 줄인다. (네트워크가 깊다) 혹은 두 가지를 결합하여 최적의 네트워크가 무엇인지 확인한 후 사용할 네트워크를 결정해야 한다.

8.1.4 앙상블을 이용한 성능 최적화

- 앙상블은 간단히 모델은 두 개 이상 섞어서 사용하는 것이다. 앙상블을 이용하는 것도 성능 향상에 도움이 된다.
- 알고리즘 튜닝을 위한 성능 최적화 방법은 하이퍼파라미터에 대한 경우의 수를 모두 고려해야 하기 때문에 모델 훈련이 수십 번에서 수백 번 필요할 수 있다. 따라서 성능 향상은 단시간에 해결되는 것이 아니고, 수많은 시행착오를 겪어야 한다.
- 성능 최적화를 위한 또 다른 방법으로는 하드웨어를 이용한 방법과 하이퍼 파라미터를 이용한 추가적인 방법들이 있다.

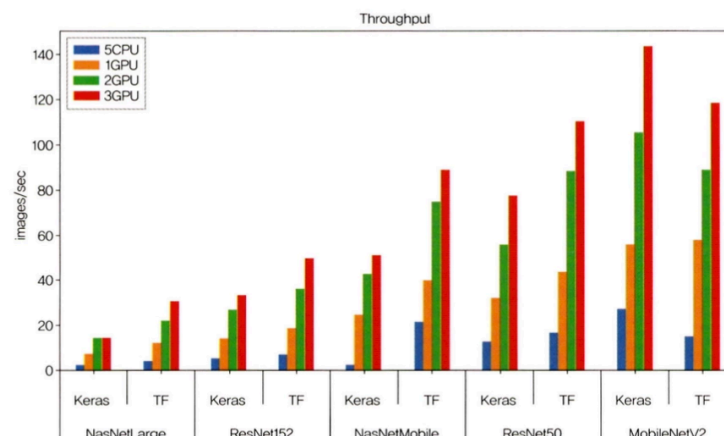
8.2 하드웨어를 이용한 성능 최적화

- 딥러닝에서 성능 최적화는 데이터 알고리즘을 이용하는 것 외에 하드웨어를 이용하는 방법이 있다. 즉, CPU가 아닌 GPU를 이용하는 것.

8.2.1 CPU와 GPU 사용의 차이

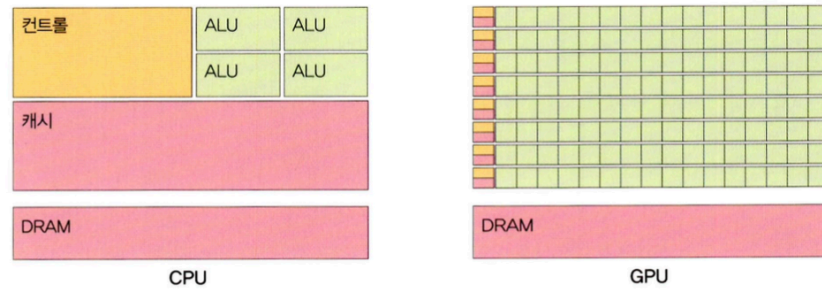
- CPU와 GPU의 성능 차이 비교 그래프

▼ 그림 8-3 CPU와 GPU 성능 비교¹



- 그래프를 살펴보면 CPU 5개를 동시에 돌려도 GPU 1개보다 성능이 좋지 못한 것을 확인할 수 있다.
- CPU는 연산을 담당하는 ALU와 명령어를 해석하고 실행하는 control, 데이터를 담아 두는 cache로 구성되어있다. CPU는 명령어가 입력되는 순서대로 데이터를 처리하는 직렬 처리 방식이다. 즉, CPU는 한 번에 하나의 명령어만 처리하기 때문에 연산을 담당하는 ALU(Arithmetic Logic Unit)의 개수가 많을 필요가 없다.
- GPU는 병렬 처리를 위해 개발되었다. 캐시 메모리 비중이 낮고, ALU 개수가 많아졌기 때문에, 여러 명령을 동시에 처리하는 병렬 처리 방식에 특화되어있다. 또한 하나의 코어에 ALU 수백~수천 개가 장착되어 있기 때문에 CPU로는 시간이 많이 걸리는 3D 그래픽 작업 등을 빠르게 수행할 수 있다.
- 개별적 코어 속도는 CPU > GPU
 - 행렬 연산에서 많이 사용하는 재귀 연산 같은 직렬 연산을 수행할 때는 CPU가 더 적합 (ex. 3*3 행렬에서 $A \rightarrow B \rightarrow C$ 열을 순차적으로 처리)

♥ 그림 8-4 CPU와 GPU의 구조 비교



- 하지만 역전파처럼 복잡한 미적분은 병렬 연산을 해야 속도가 빨라진다. A, B, C열을 얼마나 동시/에 처리하느냐에 따라 계산 시간이 달라지기 때문에 병렬 처리는 딥러닝에서 속도와 성능을 높여주는 주요 요인이 될 수 있다. 딥러닝은 데이터를 수백에서 수천만 건 까지 다루는데, 여기에서 다룬다는 것은 데이터를 벡터로 변환한 후 연산을 수행한다는 의미. 연산을 수행할 때 CPU에서 한 번에 하나의 명령어만 처리한다면 하나의 모델을 훈련시키는 데 며칠 혹은 몇 달이 걸릴 수 있다. 하지만 GPU에서 병렬로 처리할 경우 모델 훈련 시간을 많이 단축시킬 수 있기 때문에 GPU 사용은 선택이 아닌 필수!

8.2.2 GPU를 이용한 성능 최적화

- 윈도우환경에서 GPU용 파이토치를 설치하려면 CUDA와 cuDNN을 설치해야한다. CUDA는 NVIDIA에서 개발한 GPU 개발틀로 많은 양의 연산을 동시에 처리할 수 있다.
- 맥을 사용하고 있어서 설치 부분은 건너뛰

8.3 하이퍼파라미터를 이용한 성능 최적화

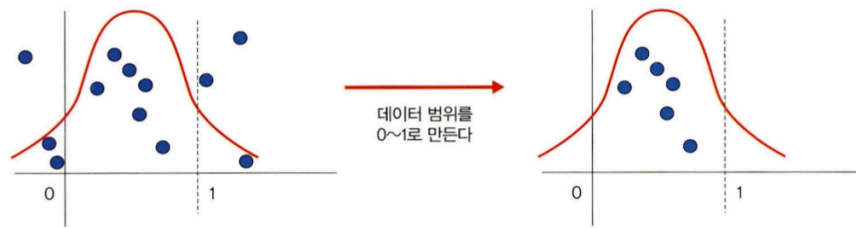
- 하이퍼파라미터를 이용한 성능 최적화의 추가적인 방법으로 배치 정규화, 드롭아웃, 조기 종료가 있다.

8.3.1 배치 정규화를 이용한 성능 최적화

정규화 (Normalization)

- 데이터 범위를 사용자가 원하는 범위로 제한하는 것. 이미지 데이터는 픽셀 정보로 0~255 사이의 값을 가지는데, 이를 255로 나누면 0~1.0 사이의 값을 가지게 된다.

♥ 그림 8-31 정규화



- 정규화는 각 특성 범위(scale)를 조정한다는 의미로 특성 스케일링(feature scaling)이라고도 한다. 스케일 조정을 위해 `MinMaxScaler()` 기법을 사용하므로 수식은 다음과 같다.

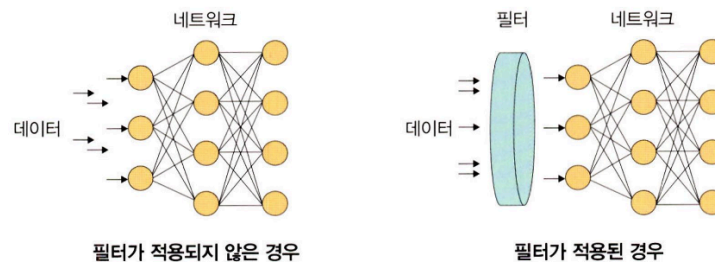
$$\frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

(x: 입력 데이터)

규제화 (Regularization)

- 규제화는 모델 복잡도를 줄이기 위해 제약을 두는 방법이다. 이때 제약은 데이터가 네트워크에 들어가기 전에 필터를 적용한 것이다.

♥ 그림 8-32 규제화



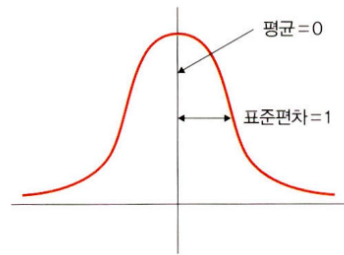
- 왼쪽 그림처럼 필터가 적용되지 않은 경우 모든 데이터가 네트워크에 투입되지만, 오른쪽 그림은 필터로 걸러진 데이터만 네트워크에 투입되어 빠르고 정확한 결과를 얻을 수 있다.
- 규제를 이용하여 모델 복잡도를 줄이는 방법으로는 드롭아웃과 조기 종료 등이 있다.

표준화 (Standardization)

- 기존 데이터를 평균은 0, 표준 편차는 1인 형태의 데이터로 만드는 방법이다. 다른 표현으로 **표준화 스칼라 (standard scaler)** 혹은 **z-스코어 정규화(z-score)**

normalization)라고도 한다.

▼ 그림 8-33 표준화



- 평균을 기준으로 얼마나 떨어져 있는지 살펴볼 때 사용된다. 데이터 분포가 가우시안 분포를 따를 때 다음 수식을 사용한다.

$$\frac{x - m}{\sigma}$$

(σ : 표준편차, x : 관측 값(입력 값), m : 평균)

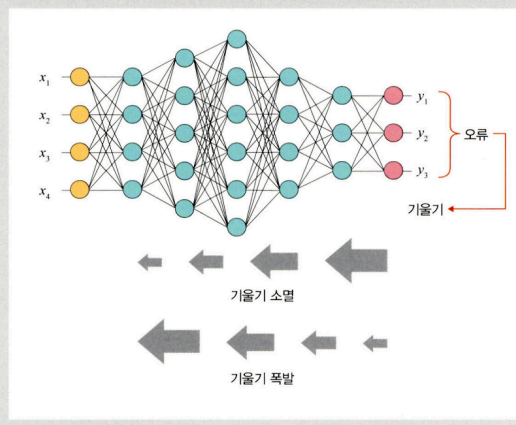
배치 정규화 (Batch Normalization)

- 2015년 "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift"라는 논문에 설명되어 있는 기법으로, 데이터 분포가 안정되어 학습 속도를 높일 수 있다.
- 배치 정규화는 기울기 소멸 (gradient vanishing)이나 기울기 폭발 (gradient exploding) 같은 문제를 해결하기 위한 방법이다. 이 문제들을 해결하기 위한 또 다른 방법으로는 손실 함수로 ReLU를 사용하거나 초기값 튜닝, 학습률 (learning rate) 등을 조정한다.

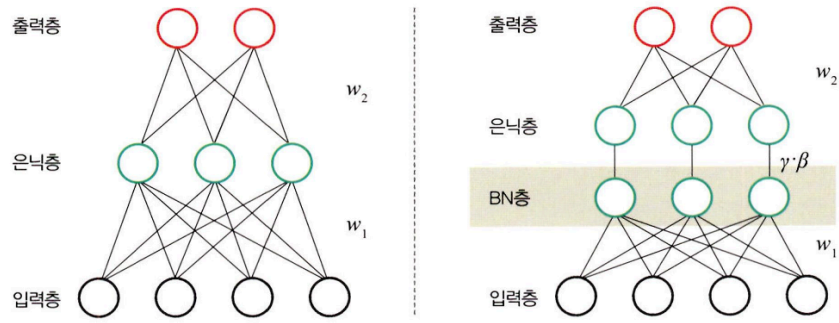
Note ≡ 기울기 소멸과 기울기 폭발

- 기울기 소멸: 오차 정보를 역전파시키는 과정에서 기울기가 급격히 0에 가까워져 학습이 되지 않는 현상입니다.
- 기울기 폭발: 학습 과정에서 기울기가 급격히 커지는 현상입니다.

▼ 그림 8-34 기울기 소멸과 기울기 폭발



♥ 그림 8-35 배치 정규화



- 배치 정규화가 소개된 논문에 따르면 기울기 소멸과 폭발의 원인은 내부 공변량 변화 (internal covariance shift) 때문인데, 이것은 네트워크의 각 층마다 활성화 함수가 적용되면서 입력 값들의 분포가 계속 바뀌는 현상이다. 따라서 분산된 분포를 정규 분포로 만들기 위해 표준화와 유사한 방식을 미니 배치에 적용해 평균은 0으로, 표준 편차는 1로 유지하도록 한다.

$$\mu\beta \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad \text{---①}$$

① 미니 배치 평균을 구합니다.

$$\sigma^2\beta \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu\beta)^2 \quad \text{---②}$$

② 미니 배치의 분산과 표준편차를 구합니다.

$$\hat{x}_i \leftarrow \frac{x_i - \mu\beta}{\sqrt{\sigma^2\beta + \epsilon}} \quad \text{---③}$$

③ 정규화를 수행합니다.

$$y_i \leftarrow \gamma \hat{x}_i + \beta \Leftrightarrow BN_{\gamma, \beta}(x_i) \quad \text{---④}$$

④ 스케일(scale)을 조정(데이터 분포 조정)합니다.

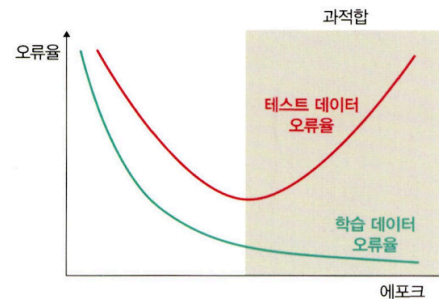
$$\left(\begin{array}{l} \text{입력: } \beta = \{x_1, x_2, \dots, x_n\} \\ \text{학습해야 할 하이퍼파라미터: } \gamma, \beta \\ \text{출력: } y_i = BN_{\gamma, \beta}(x_i) \end{array} \right)$$

- 매 단계마다 활성화 함수를 거치면서 데이터셋 분포가 일정해지기 때문에 속도를 향상시킬 수 있지만 단점도 존재한다.
 - 배치 크기가 작을 때는 정규화 값이 기존 값과 다른 방향으로 훈련될 수 있다. 예를 들어 분산이 0이면 정규화 자체가 안되는 경우가 생길 수 있다.
 - RNN은 네트워크 계층별로 미니 정규화를 적용해야 하기 때문에 모델이 더 복잡해지면서 비효율적일 수 있음
- 이런 문제들을 해결하기 위한 가중치 수정, 네트워크 구성 변경 등을 수행하지만, 무엇보다 중요한 것은 배치 정규화를 적용하면 적용하지 않았을 때보다 성능이 좋아지기 때문에 많이 사용됨.

8.3.2 드롭아웃을 이용한 성능 최적화

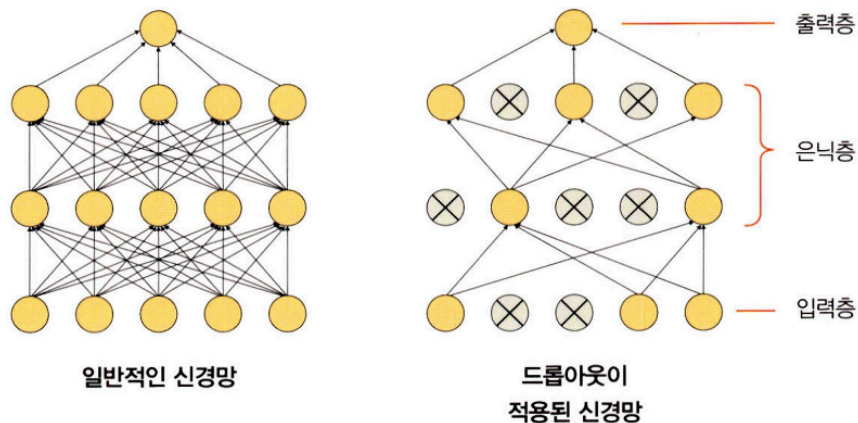
- 과적합은 훈련 데이터셋을 과하게 학습하는 것을 의미한다. 훈련 데이터셋에 대해 훈련을 계속한다면 오류는 줄어들지만 테스트 데이터셋에 대한 오류는 어느 순간부터 증가하는데, 이러한 모델을 과적합되어 있다고 한다.

▼ 그림 8-36 훈련과 오류율



- 드롭아웃 (dropout)이란 훈련할 때 일정 비율의 뉴런만 사용하고, 나머지 뉴런에 해당하는 가중치는 업데이트하지 않는 방법이다. (매 단계마다 사용하지 않는 뉴런을 바꾸어가며 훈련시킨다.) 즉, 드롭 아웃은 노드를 임의로 끄면서 학습하는 방법으로, 은닉층에 배치된 노드 중 일부를 임의로 끄면서 학습한다. 꺼진 노드는 신호를 전달하지 않으므로 지나친 학습을 방지하는 효과가 생긴다.

▼ 그림 8-37 드롭아웃



- 왼쪽은 일반적인 신경망이고, 오른쪽은 드롭아웃이 적용된 신경망의 모습이다. 일부 노드들은 비활성화되고 남은 노드들로 신호가 연결되는 신경망 형태를 띤다. 어떤 노드를 비활성화할지는 학습할 때마다 무작위로 선정되며, 테스트 데이터로 평가할 때는 노드들을 모두 사용하여 출력하되 노드 삭제 비율을 곱해서 성능을 평가한다.
- 드롭아웃을 사용하면 훈련 시간이 길어지는 단점이 있지만, 모델 성능을 향상하기 위해 자주 쓰인다.
- 배치 정규화와 드롭 아웃에 대한 파이토치 예제 (FashionMNIST 데이터 사용)

8-1 라이브러리 호출

```
import torch
import matplotlib.pyplot as plt
import numpy as np

import torchvision
import torchvision.transforms as transforms

import torch.nn as nn
import torch.optim as optim
```

8-2 데이터셋 내려받기

```
trainset = torchvision.datasets.FashionMNIST(
    root='data/', train = True, download = True,
    transform = transforms.ToTensor()
)
```

8-3 데이터셋을 메모리로 가져오기

```
batch_size = 4 # 한 번에 4 개씩 쪼개서 데이터를 가져옴
trainloader = torch.utils.data.DataLoader(trainset, batch_size = batch_size,
                                           shuffle = True)
```

8-4 데이터셋 분리

```
dataiter = iter(trainloader)
images, labels = next(dataiter)

print(images.shape)
print(images[0].shape)
print(labels[0].item())
```

```
torch.Size([4, 1, 28, 28])
torch.Size([1, 28, 28])
6
```

- `torch.Size([4, 1, 28, 28])`
 - 4: batch size
 - 1: channel (흑백 이미지 = 1, 컬러 = 3)
 - 28, 28: 너비, 높이 (픽셀 크기의 이미지)

8-5 이미지 데이터를 출력하기 위한 전처리

```
def imshow(img, title):
    plt.figure(figsize = (batch_size * 4, 4))
    plt.axis('off')
    plt.imshow(np.transpose(img, (1, 2, 0)))
    plt.title(title)
    plt.show()
```

- 파이토치는 이미지 데이터셋을 [batch size, channel, width, height] 순서대로 저장한다. 이를 matplotlib으로 출력하기 위해서는 이미지가 [width, height, channel] 형태여야하기 때문에 `transpose()` 로 데이터 형태를 변경한다.

8-6 이미지 데이터 출력 함수

```
def show_batch_images(dataloader):
    # image 크기: (4, 28, 28, 1); (batch size, height, width, channel)
    images, labels = next(iter(dataloader))

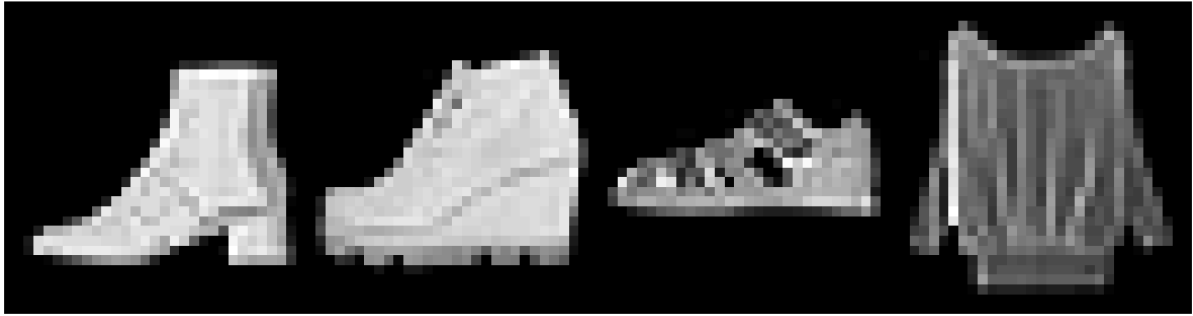
    # 좌표에 이미지 픽셀을 대응시켜 그리드 형태로 출력
    img = torchvision.utils.make_grid(images)
    # imshow 함수를 사용함으로써 데이터 형태가 (channel, height, width) -> (height, width, channel)
    imshow(img, title=[str(x.item()) for x in labels])

    return images, labels
```

8-7 이미지 출력

```
images, labels = show_batch_images(trainloader)
```

['9', '9', '5', '6']



```
classes = {
    0: "T-Shirt/Top",
    1: "Trouser",
    2: "Pullover",
    3: "Dress",
    4: "Coat",
    5: "Sandal",
    6: "Shirt",
    7: "Sneaker",
    8: "Bag",
    9: "Ankle Boot"
}
```

- 이미지 위의 숫자는 레이블을 의미
- batch size를 4로 설정했기 때문에 한 번에 4 개의 이미지만 가져옴

8-8 배치 정규화가 적용되지 않은 네트워크

```
class NormalNet(nn.Module):
    def __init__(self):
        super(NormalNet, self).__init__()
        self.classifier = nn.Sequential(
            nn.Linear(784, 48), # (28 * 28) = 784 → 48 nodes
            nn.ReLU(),
            nn.Linear(48, 24), # 48 → 24 nodes
            nn.ReLU(),
            nn.Linear(24, 10) # FashionMNIST: 10 classes
        )
    def forward(self, x):
        x = x.view(x.size(0), -1)
```



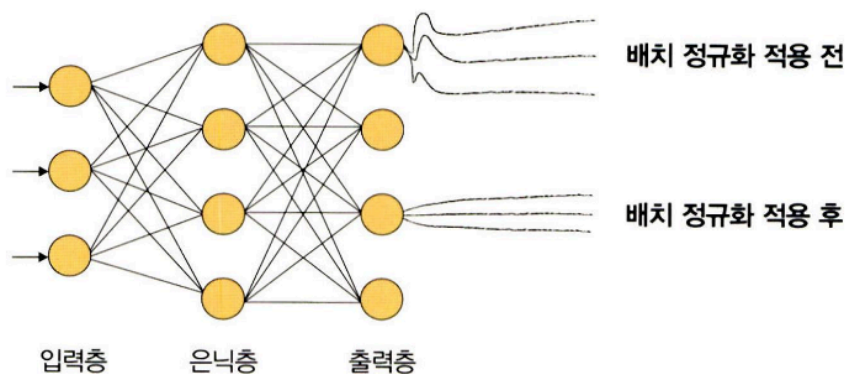
```
x = self.classifier(x) # nn.Sequential에서 정의한 계층 호출
return x
```

8-9 배치 정규화가 포함된 네트워크

```
class BNNet(nn.Module):
    def __init__(self):
        super(BNNet, self).__init__()
        self.classifier = nn.Sequential(
            nn.Linear(784, 48), # (28 * 28) = 784 → 48 nodes
            nn.BatchNorm1d(48),
            nn.ReLU(),
            nn.Linear(48, 24), # 48 → 24 nodes
            nn.BatchNorm1d(24),
            nn.ReLU(),
            nn.Linear(24, 10) # FashionMNIST: 10 classes
        )
    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = self.classifier(x) # nn.Sequential에서 정의한 계층 호출
        return x
```

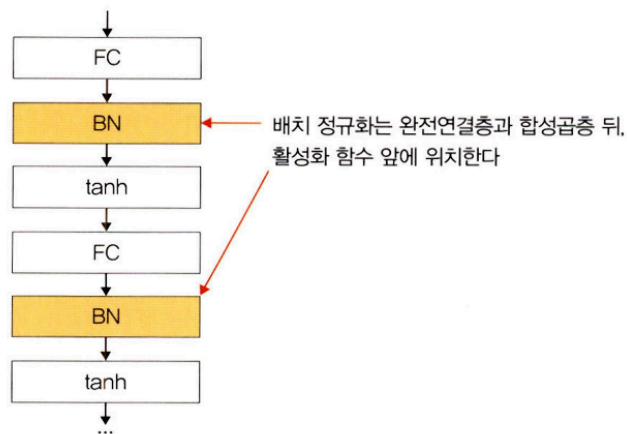
- **BatchNorm1d** 에서 사용되는 파라미터는 특성 개수로 **이전 계층의 출력 채널**이 된다.
 - 배치 정규화로 입력 분포를 고르게 맞춰 줄 수 있음

▼ 그림 8-39 배치 정규화



- 배치 정규화는 완전 연결층과 합성곱층 뒤, 활성화 함수 앞에 위치

▼ 그림 8-40 배치 정규화 위치



8-10 배치 정규화가 적용되지 않은 모델 선언

```
model = NormalNet()
print(model)
```

```
NormalNet(
  (classifier): Sequential(
    (0): Linear(in_features=784, out_features=48, bias=True)
    (1): ReLU()
    (2): Linear(in_features=48, out_features=24, bias=True)
    (3): ReLU()
    (4): Linear(in_features=24, out_features=10, bias=True)
  )
)
```

8-11 배치 정규화가 적용된 모델 선언

```
model_bn = BNNet()
print(model_bn)
```

```
BNNet(
  (classifier): Sequential(
    (0): Linear(in_features=784, out_features=48, bias=True)
    (1): BatchNorm1d(48, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): Linear(in_features=48, out_features=24, bias=True)
    (4): BatchNorm1d(24, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): ReLU()
    (6): Linear(in_features=24, out_features=10, bias=True)
  )
)
```

8-12 데이터셋 메모리로 불러오기

```
batch_size = 512 # batch size 변경
trainloader = torch.utils.data.DataLoader(trainset,
                                           batch_size = batch_size, shuffle = True)
```

8-13 옵티마이저, 손실 함수 정의

```
loss_fn = nn.CrossEntropyLoss()
opt = optim.SGD(model.parameters(), lr = 0.01)
opt_bn = optim.SGD(model_bn.parameters(), lr = 0.01)
```

8-14 모델 학습

```
loss_arr = []
loss_bn_arr = []
max_epochs = 2

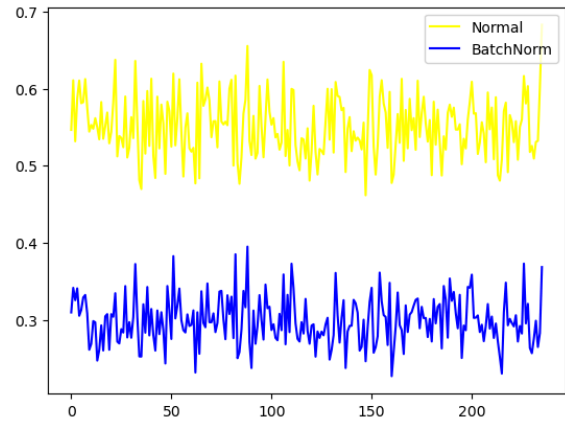
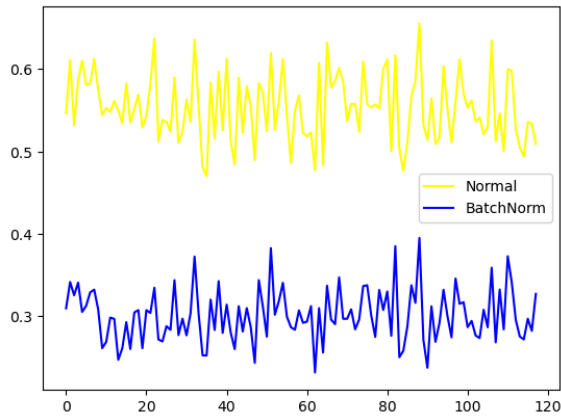
for epoch in range(max_epochs):
    for i, data in enumerate(trainloader, 0):
        inputs, labels = data
        opt.zero_grad() # BN 적용 x model train
        outputs = model(inputs)
        loss = loss_fn(outputs, labels)
        loss.backward()
        opt.step()

        opt_bn.zero_grad() # BN 적용 model train
        outputs_bn = model_bn(inputs)
        loss_bn = loss_fn(outputs_bn, labels)
        loss_bn.backward()
        opt_bn.step()

        loss_arr.append(loss.item())
        loss_bn_arr.append(loss_bn.item())

plt.plot(loss_arr, 'yellow', label = 'Normal')
plt.plot(loss_bn_arr, 'blue', label = 'BatchNorm')
```

```
plt.legend()
plt.show()
```



- 배치 정규화가 적용된 모델과 적용되지 않은 모델 모두 시간이 흐를 수록 오차가 줄어들지만, 오차가 줄어드는 범위 및 값의 차이는 명백히 다르다. 배치 정규화가 적용된 모델의 경우 더 낮은 값으로 안정적인 범위 내에서 줄어들고 있다. 즉, 배치 정규화가 적용된 모델은 에포크가 진행될 수록 오차도 줄어들면서 안정적인 학습을 하고 있다.

8-15 데이터셋 분포를 출력하기 위한 전처리

N = 50

noise = 0.3

```
x_train = torch.unsqueeze(torch.linspace(-1, 1, N), 1)
```

```
y_train = x_train + noise * torch.normal(torch.zeros(N, 1), torch.ones(N, 1))
```

```
x_test = torch.unsqueeze(torch.linspace(-1, 1, N), 1)
```

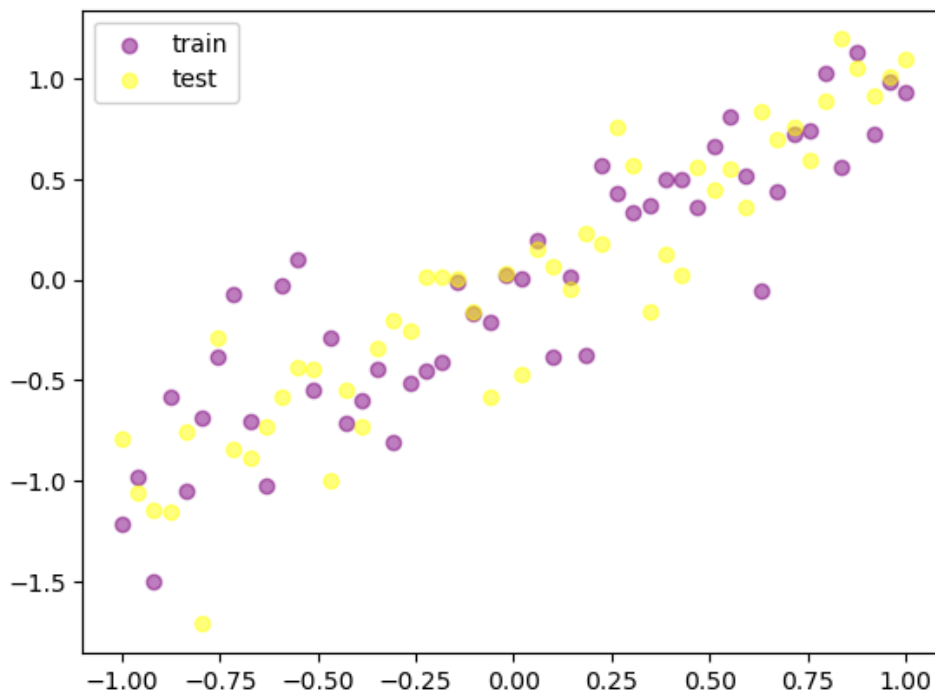
```
y_test = x_test + noise * torch.normal(torch.zeros(N, 1), torch.ones(N, 1))
```

- `torch.unsqueeze(torch.linspace(-1, 1, N), 1)` : 훈련 데이터셋이 -1~1의 값을 갖도록 조정한다.
 - `torch.linspace` 는 주어진 범위에서 균등한 값을 갖는 텐서를 만들기 위해 사용된다.
 - `torch.linspace(-1, 1, N)` : -1~1의 범위에서 N개의 균등한 값을 갖는 텐서를 생성한다.
 - `torch.unsqueeze()` : 차원을 늘리기 위해 사용. `torch.unsqueeze(torch.linspace(-1, 1, N), 1)` : `torch.linspace(-1, 1, N)` 의 첫 번째 자리에 차원을 증가시킨다.
- `y_train = x_train + noise * torch.normal(torch.zeros(N, 1), torch.ones(N, 1))`

- 훈련 데이터셋 값의 범위가 정규 분포를 갖도록 조정한다.
- `torch.normal` : 정규분포로부터 무작위 표본 추출을 위해 사용한다.

8-16 데이터 분포를 그래프로 출력

```
plt.scatter(x_train.data.numpy(), y_train.data.numpy(), c= 'purple',
            alpha=0.5, label = 'train')
plt.scatter(x_test.data.numpy(), y_test.data.numpy(), c= 'yellow',
            alpha=0.5, label = 'test')
plt.legend()
plt.show()
```



8-17 드롭아웃을 위한 모델 생성

```
N_h = 100
model = torch.nn.Sequential(
    torch.nn.Linear(1, N_h),
    torch.nn.ReLU(),
    torch.nn.Linear(N_h, N_h),
    torch.nn.ReLU(),
    torch.nn.Linear(N_h, 1)
) # Dropout x
```

```

model_dropout = torch.nn.Sequential(
    torch.nn.Linear(1, N_h),
    torch.nn.Dropout(0.2),
    torch.nn.ReLU(),
    torch.nn.Linear(N_h, N_h),
    torch.nn.Dropout(0.2),
    torch.nn.ReLU(),
    torch.nn.Linear(N_h, 1)
) # Dropout o

```

8-18 옵티마이저와 손실 함수 지정

```

opt = torch.optim.Adam(model.parameters(), lr = 0.01)
opt_dropout = torch.optim.Adam(model_dropout.parameters(), lr = 0.01)
loss_fn = torch.nn.MSELoss()

```

8-19 모델 학습

```

max_epochs = 1000
for epoch in range(max_epochs):
    pred = model(x_train)
    loss = loss_fn(pred, y_train)
    opt.zero_grad()
    loss.backward()
    opt.step()

    pred_dropout = model_dropout(x_train)
    loss_dropout = loss_fn(pred_dropout, y_train)
    opt_dropout.zero_grad()
    loss_dropout.backward()
    opt_dropout.step()

    if epoch % 50 == 0:
        model.eval()
        model_dropout.eval()

        test_pred = model(x_test)

```

```

test_loss = loss_fn(test_pred, y_test)

test_pred_dropout = model_dropout(x_test)
test_loss_dropout = loss_fn(test_pred_dropout, y_test)

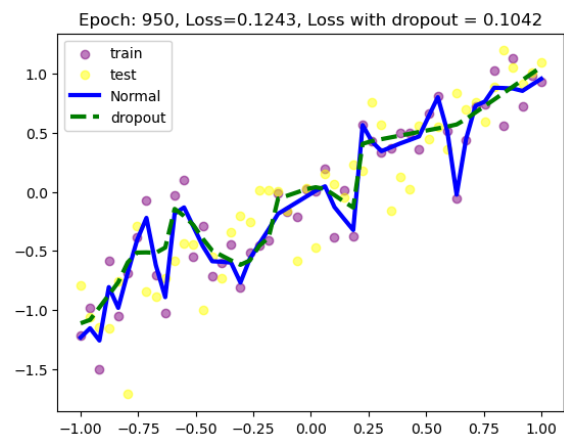
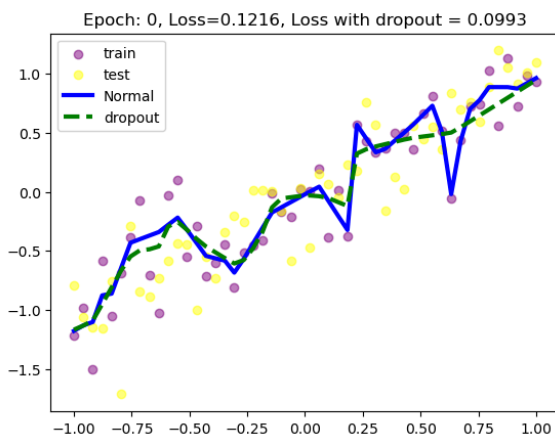
plt.scatter(x_train.data.numpy(), y_train.data.numpy(), c= 'purple',
            alpha = 0.5, label = 'train')
plt.scatter(x_test.data.numpy(), y_test.data.numpy(), c= 'yellow',
            alpha = 0.5, label = 'test')

plt.plot(x_test.data.numpy(), test_pred.data.numpy(), 'b-',
         lw = 3, label = 'Normal')
plt.plot(x_test.data.numpy(), test_pred_dropout.data.numpy(), 'g--',
         lw = 3, label = 'dropout')

plt.title(f'Epoch: {epoch}, Loss={test_loss}, Loss with dropout = {test_loss_dropout}')

plt.legend()
model.train()
model_dropout.train()
plt.pause(0.05)

```



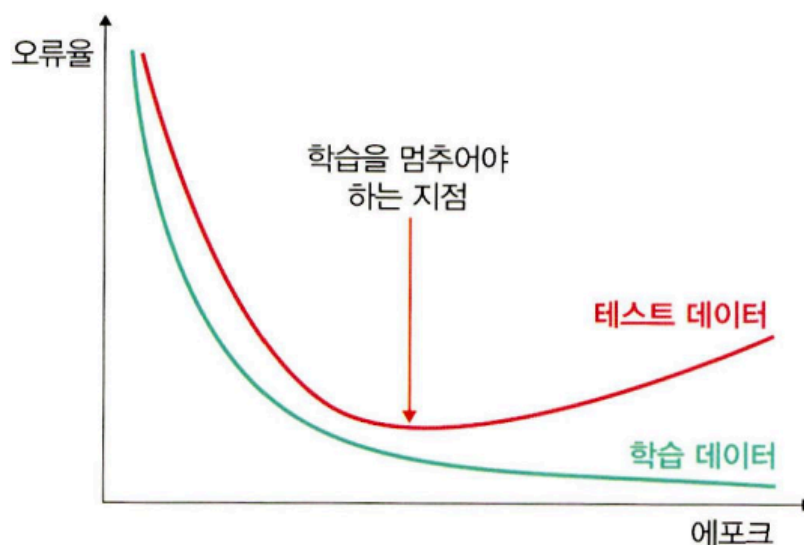
- 전반적으로 오차가 줄어드는 범위는 크지 않지만 드롭아웃을 적용했을 때의 오차가 더 낮다. 출력 결과에서 초록색 점선과 파란색 실선의 차이가 크지 않아 보이지만 실제로는 큰 차이가 존재한다. 훈련 횟수가 늘어날 수록 파란색 실선은 가장 자리의 자주색 점들을 찾아가고 있다. 문제는 자주색이 훈련 데이터셋을 의미한다는 것이고, 이는 과적합 현상을 보이고 있다는 것이다. 과적합이 발생하는 모델은 훈련 데이터에 대한 정확도는 높을

수 있지만 새로운 데이터, 즉 검증 데이터나 테스트 데이터에 대해서는 제대로 동작하지 않는 문제가 있다. 이와 같이 과적합 현상을 방지하기 위해 드롭아웃을 사용하며, 초록색 점선 그래프는 과적합이 나타나지 않는 것을 확인할 수 있다.

8.3.3 조기 종료를 이용한 성능 최적화

- 조기 종료는 뉴럴 네트워크가 과적합을 회피하는 규제 기법이다. 훈련 데이터와 별도로 검증 데이터를 준비하고, 매 에포크마다 검증 데이터에 대한 오차를 측정하여 모델의 종료 시점을 제어한다. 즉, 과적합이 발생하기 전까지 학습에 대한 오차와 검증에 대한 오차 모두 감소하지만, 과적합이 발생하면 훈련 데이터셋에 대한 오차는 감소하는 반면 검증 데이터셋에 대한 오차는 증가한다. 따라서 조기 종료는 검증 데이터셋에 대한 오차가 증가하는 시점에서 학습을 멈추도록 조정한다.
- 하지만 조기 종료는 학습을 언제 종료시킬지 결정할 뿐, 최고의 성능을 갖는 모델을 보장하지는 않는다는 점을 주의해야 한다.

▼ 그림 8-50 조기 종료



8-20 라이브러리 호출

```
import torch
import matplotlib
import matplotlib.pyplot as plt
import numpy as np

import torchvision
```



```

import torchvision.models as models # 사전 학습된 모델을 이용하고자 할 때 사용
from torchvision import transforms, datasets

import torch.nn as nn
import torch.optim as optim

import time
import argparse
from tqdm import tqdm
matplotlib.style.use('ggplot')
device = torch.device('mps' if torch.backends.mps.is_available() else 'cpu')

```

8-21 데이터셋 전처리

```

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean = [0.485, 0.456, 0.406],
                          std = [0.229, 0.224, 0.225])
])
val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean = [0.485, 0.456, 0.406],
                          std = [0.229, 0.224, 0.225])
])

```

- 예제에서 사용하는 데이터셋은 핫도그와 핫도그가 아닌 이미지들을 사용한다.

♥ 그림 8-51 핫도그 이미지



♥ 그림 8-52 핫도그가 아닌 이미지



8-22 데이터셋 가져오기

```
train_dataset = datasets.ImageFolder(  
    root = r'archive/train',  
    transform = train_transform  
)  
train_dataloader = torch.utils.data.DataLoader(  
    train_dataset, batch_size = 16, shuffle = True  
)  
val_dataset = datasets.ImageFolder(  
    root = r'archive/test',  
    transform = val_transform  
)  
val_dataloader = torch.utils.data.DataLoader(  
    val_dataset, batch_size = 16, shuffle = True  
)
```

8-23 모델 생성

```
def resnet50(pretrained = True, requires_grad = False):  
    model = models.resnet50(progress = True, pretrained = pretrained)
```

```

if requires_grad == False: # parameter 고정 -> 역전파 중 기울기 계산 x
    for param in model.parameters():
        param.requires_grad = False
elif requires_grad == True: # parameter 값이 역전파 중에 기울기 계산에 반영됨
    for param in model.parameters():
        param.requires_grad = True
model.fc = nn.Linear(2048, 2) # 분류
return model

```

8-24 학습률 감소

- 파이토치에서 조기 종료와 함께 자주 사용되는 것으로 학습률 감소가 있다. 학습률에 대한 값을 고정시켜서 모델을 학습시키는 것이 아니라 학습이 진행되는 과정에서 학습률을 조금씩 낮춰 주는 성능 튜닝 기법. 학습률 감소는 학습률 스케줄러를 이용하는데, 주어진 'patience' 횟수만큼 검증 데이터셋에 대한 오차 감소가 없으면 주어진 'factor'만큼 학습률을 감소시켜서 모델 학습의 최적화가 가능하도록 도와준다.

```

class LRScheduler():
    def __init__(
        self, optimizer, patience = 5, min_lr = 1e-6, factor = 0.5
    ):
        self.optimizer = optimizer
        self.patience = patience
        self.min_lr = min_lr
        self.factor = factor
        self.lr_scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
            self.optimizer,
            mode = 'min',
            patience = self.patience,
            factor = self.factor,
            min_lr = self.min_lr,
            verbose = True
        )
    def __call__(self, val_loss):
        self.lr_scheduler.step(val_loss)

```

- `def __init__()` : 학습 과정에서 모델 성능에 대한 개선이 없을 경우 학습률 값을 조절하여 모델의 개선을 유도하는 콜백 함수

- `lr_scheduler.ReduceLROnPlateau()`: 검증 데이터셋에 대한 오차의 변동이 없으면 학습률을 `factor` 배로 감소시킴
- `optimizer`: 파라미터를 갱신시키는 부분
- `mode`: 언제 학습률을 조정할지에 대한 기준이 되는 값. 만약 검증 데이터셋에 대한 오차를 기준으로 사용하면 오차가 더 이상 감소되지 않을 때 학습률을 조정함. 이때 오차값이 최소가 되어야하는지 최대가 되어야하는지 알려주는 파라미터가 `mode` 이다.
 - 학습률 조정의 기준이 되는 값을 모델의 정확도로 사용하면 값이 클수록 좋기 때문에 `max`를 지정
 - 모델의 오차를 사용할 경우 작을 수록 좋기 때문에 `min`을 지정
- `verbose`: 조기 종료의 시작과 끝을 출력하기 위해 사용. 1로 설정할 경우 조기 종료가 적용되면 종료되었다고 화면에 출력되지만, 0으로 설정할 경우 아무런 출력 없이 학습을 종료함
- `self.lr_scheduler.step(val_loss)`: 실제로 학습률을 업데이트. 에포크 단위로 검증 데이터셋에 대한 오차를 받아서 이전 오차와 비교했을 때 차이가 없다면 학습률을 업데이트한다.

8-25 조기 종료

```
class EarlyStopping():
    def __init__(self, patience=5, verbose=False, delta=0, path='checkpoint.pt'):
        self.patience = patience
        self.verbose = verbose
        self.counter = 0
        self.best_score = None      # 검증 데이터셋에 대한 오차 최적화 값(오차가 가?
        self.early_stop = False    # 조기 종료를 의미하며 초기값은 False로 설정
        self.val_loss_min = np.Inf  # np.Inf(infinity)는 넘파이에서 무한대를 표현
        self.delta = delta
        self.path = path

    def __call__(self, val_loss, model): # 에포크만큼 학습이 반복되면서 best_loss가
        # best_loss에 진전이 없으면 조기 종료 후 모델을 저장
        score = -val_loss

        if self.best_score is None:    # best_score에 값이 존재하지 않으면 실행
            self.best_score = score
            self.save_checkpoint(val_loss, model)
```

```

elif score < self.best_score + self.delta: # best_score + delta가 score보다
    self.counter += 1
    print(f'EarlyStopping counter: {self.counter} out of {self.patience}')
    if self.counter >= self.patience:
        self.early_stop = True

else: # 그 외 모든 경우에 실행
    self.best_score = score
    self.save_checkpoint(val_loss, model)
    self.counter = 0

def save_checkpoint(self, val_loss, model):
    if self.verbose:
        print(f'Validation loss decreased ({self.val_loss_min:.6f}) → {val_loss:.6f}')
    torch.save(model.state_dict(), self.path)
    self.val_loss_min = val_loss

```

- **delta**: 오차가 개선되고 있다고 판단하기 위한 최소 변화량. 변화량이 delta보다 적으면 개선이 없다고 판단한다.
- 케라스에서 제공하는 콜백을 이용하면 쉽게 조기 종료를 구현할 수 있지만, 파이토치에서는 조기 종료 함수를 사용자가 직접 구현해야함

```

from keras.callbacks import ModelCheckpoint, EarlyStopping
checkpoint = ModelCheckpoint('checkpoint-epoch.h5'.format(EPOCH, BATCH_SIZE), ----- 파일명 지정
                            monitor='val_loss', ----- val_loss 값이 개선되었을 때 호출
                            verbose=1, ----- 로그 출력
                            save_best_only=True, ----- 가장 최적의 값만 저장
                            mode='auto' ----- auto 의미는 시스템이 알아서 best 값을 찾으라는 것
                            )

earlystopping = EarlyStopping(monitor='val_loss', ----- 학습률 업데이트 기준 설정(val_loss)
                              patience=10 ----- 에포크가 진행되는 열 번 동안 모델의 오차가
                              ) ----- 개선되지 않는다면 종료

```

8-26 인수값 지정

- 예제에서는 학습률 감소와 조기 종료 두 개에 대한 성능 튜닝을 진행한다. 즉, 함수에 넘겨주는 인수(argument) 값에 따라 다른 동작을 하도록 해야하는데, 이때 **argparse** 라이브러리

브러리를 사용한다. `ArgumentParser()` 를 이용하여 변수와 타입을 정의해주고 `add_argument()` 를 이용해서 변수에 인수 값을 하나씩 추가한다. 마지막으로 `parse_args` 를 통해 사용자로 부터 입력 받은 값들을 `args` 에 저장한다.

```
parser = argparse.ArgumentParser()
parser.add_argument('--lr-scheduler', dest = 'lr_scheduler', action = 'store_true')
parser.add_argument('--early-stopping', dest = 'early_stopping', action = 'store_true')
args, unknown = parser.parse_known_args()
args = vars(args)
```

8-27 사전 훈련된 모델의 파라미터 확인

```
print(f"Computation device: {device}\n")
model = models.resnet50(pretrained = True).to(device)
total_params = sum(p.numel() for p in model.parameters())
print(f"{total_params} total parameters.")

total_trainable_params = sum(
    p.numel() for p in model.parameters() if p.requires_grad
)
print(f"{total_trainable_params} training parameters.")
```

Computation device: mps

```
/opt/anaconda3/envs/euron/lib/python3.9/site-packages/torchvision/models/_utils.py:208:
warnings.warn(
/opt/anaconda3/envs/euron/lib/python3.9/site-packages/torchvision/models/_utils.py:223:
warnings.warn(msg)
25557032 total parameters.
25557032 training parameters.
```

8-28 옵티마이저와 손실 함수 지정

```
lr = 0.001
epochs = 100
optimizer = torch.optim.Adam(model_dropout.parameters(), lr = 0.01)
criterion = nn.CrossEntropyLoss()
```

8-29 오차, 정확도 및 모델의 이름에 대한 문자열

```
loss_plot_name = 'loss' # 오차 출력에 대한 문자열
acc_plot_name = 'accuracy' # 정확도 출력에 대한 문자열
model_name = 'model' # 모델을 지정하기 위한 문자열
```

8-30 오차, 정확도 및 모델의 이름에 대한 문자열

- '—lr-scheduler' 또는 '—early-stopping' 인수를 사용할 경우 오차, 정확도 및 모델의 이름으로 사용할 문자열을 지정

```
if args['lr_scheduler']:
    print('INFO: Initializing learning rate scheduler')
    lr_scheduler = LRScheduler(optimizer)
    loss_plot_name = 'lrs_loss'
    acc_plot_name = 'lrs_accuracy'
    model_name = 'lrs_model'
if args['early_stopping']:
    print('INFO: Initializing early stopping')
    early_stopping = EarlyStopping()
    loss_plot_name = 'es_loss'
    acc_plot_name = 'es_accuracy'
    model_name = 'es_model'
```

8-31 모델 학습 함수

```
def training(model, train_dataloader, train_dataset, optimizer, criterion):
    print('Training')
    model.train()
    train_running_loss = 0.0
    train_running_correct = 0
    counter = 0
    total = 0
    prog_bar = tqdm(enumerate(train_dataloader),
                    total=int(len(train_dataset)/train_dataloader.batch_size)) # 훈련 진

    for i, data in prog_bar:
        counter += 1
        data, target = data[0].to(device), data[1].to(device)
        total += target.size(0)
```

```

optimizer.zero_grad()
outputs = model(data)
loss = criterion(outputs, target)
train_running_loss += loss.item()
_, preds = torch.max(outputs.data, 1)
train_running_correct += (preds == target).sum().item()
loss.backward()
optimizer.step()

```

```

train_loss = train_running_loss / counter
train_accuracy = 100. * train_running_correct / total
return train_loss, train_accuracy

```

8-32 모델 검정 함수

```

def validate(model, test_dataloader, val_dataset, criterion):
    print('Validating')
    model.eval()
    val_running_loss = 0.0
    val_running_correct = 0
    counter = 0
    total = 0
    prog_bar = tqdm(enumerate(test_dataloader),
                    total=int(len(val_dataset)/test_dataloader.batch_size)) # 모델 검증

    with torch.no_grad():
        for i, data in prog_bar:
            counter += 1
            data, target = data[0].to(device), data[1].to(device)
            total += target.size(0)
            outputs = model(data)
            loss = criterion(outputs, target)

            val_running_loss += loss.item()
            _, preds = torch.max(outputs.data, 1)
            val_running_correct += (preds == target).sum().item()

    val_loss = val_running_loss / counter

```



```
val_accuracy = 100. * val_running_correct / total
return val_loss, val_accuracy
```

8-33 모델 학습

```
train_loss, train_accuracy = [], [] # 훈련 데이터셋을 이용한 결과 저장 리스트
val_loss, val_accuracy = [], []    # 검증 데이터셋을 이용한 결과 저장 리스트

start = time.time()
for epoch in range(epochs):
    print(f"Epoch {epoch+1} of {epochs}")

    train_epoch_loss, train_epoch_accuracy = training(
        model, train_dataloader, train_dataset, optimizer, criterion
    )

    val_epoch_loss, val_epoch_accuracy = validate(
        model, val_dataloader, val_dataset, criterion
    )

    train_loss.append(train_epoch_loss)
    train_accuracy.append(train_epoch_accuracy)
    val_loss.append(val_epoch_loss)
    val_accuracy.append(val_epoch_accuracy)

    if args['lr_scheduler']: # 인수 값이 lr_scheduler이면 실행
        lr_scheduler(val_epoch_loss)

    if args['early_stopping']: # 인수 값이 early_stopping이면 실행
        early_stopping(val_epoch_loss, model)
        if early_stopping.early_stop:
            break

    print(f"Train Loss: {train_epoch_loss:.4f}, Train Acc: {train_epoch_accuracy:.4f}")
    print(f"Val Loss: {val_epoch_loss:.4f}, Val Acc: {val_epoch_accuracy:.2f}")

end = time.time()
print(f"Training time: {(end-start)/60:.3f} minutes")
```

다음은 모델 학습 결과입니다.

```
Epoch 1 of 100
Training
Validating
Train Loss: 2.1123, Train Acc: 60.04
Val Loss: 13.3437, Val Acc: 55.20
Epoch 2 of 100
Training
Validating
Train Loss: 0.5598, Train Acc: 73.90
Val Loss: 0.6033, Val Acc: 68.40
... 중간 생략 ...
Epoch 99 of 100
Training
Validating
Train Loss: 0.0340, Train Acc: 99.20
Val Loss: 1.5805, Val Acc: 68.60
Epoch 100 of 100
Training
Validating
Train Loss: 0.0394, Train Acc: 98.80
Val Loss: 1.0129, Val Acc: 76.00
Training time: 654.208 minutes
```

8-34 모델 학습 결과 출력

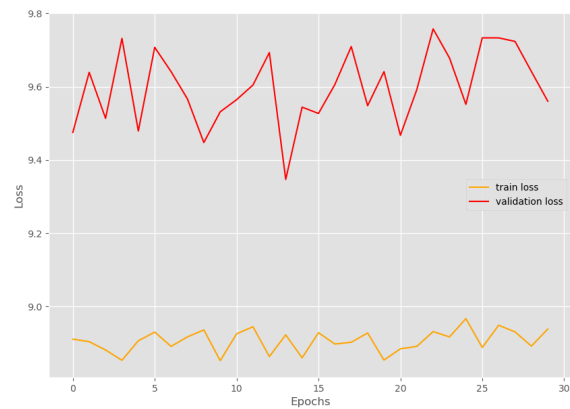
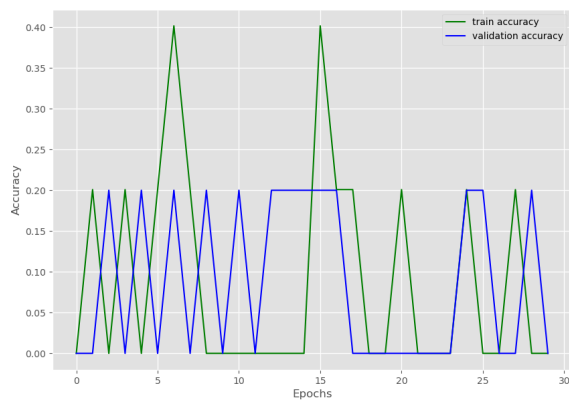
```
print('Saving loss and accuracy plots...')

plt.figure(figsize=(10, 7))
plt.plot(train_accuracy, color='green', label='train accuracy') # 훈련 데이터셋에
plt.plot(val_accuracy, color='blue', label='validation accuracy') # 검증 데이터셋
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.savefig(f"img/{acc_plot_name}.png")
plt.show()

plt.figure(figsize=(10, 7))
plt.plot(train_loss, color='orange', label='train loss') # 훈련 데이터셋에 대한 오차
plt.plot(val_loss, color='red', label='validation loss') # 검증 데이터셋에 대한 오차
```

```
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.savefig(f"img/{loss_plot_name}.png")
plt.show()

print('Saving model...')
torch.save(model.state_dict(), f"img/{model_name}.pth") # 모델을 저장
print('TRAINING COMPLETE')
```



- 작성된 코드를 파이썬으로 저장 (확인해야할 인자가 2개이기 때문에)

1. 어떤 인수도 적용하지 않았을 때 실행 결과

```
python es-python_82.py
```

Computation device: cpu

25,557,032 total parameters.

25,557,032 training parameters.

Epoch 1 of 100

Training

0%|

| 0/15 [00:00<?, ?it/s]e:\Anaconda3\envs\pytorch\lib\site-packages\torch\nn\functional.py:718: UserWarning: Named tensors and all their associated APIs are an experimental feature and subject to change. Please do not use them for anything important until they are released as stable. (Triggered internally at ..\c10\core\TensorImpl.h:1156.)

return torch.max_pool2d(input, kernel_size, stride, padding, dilation, ceil_mode)

16it [04:41, 17.62s/it]

Validating

16it [01:30, 5.64s/it]

Train Loss: 2.2272, Train Acc: 58.84

Val Loss: 7.0938, Val Acc: 43.60

Epoch 2 of 100

Training

16it [04:39, 17.45s/it]

Validating

16it [01:30, 5.66s/it]

Train Loss: 0.5848, Train Acc: 71.69

Val Loss: 2.0658, Val Acc: 54.80

Epoch 3 of 100

Training

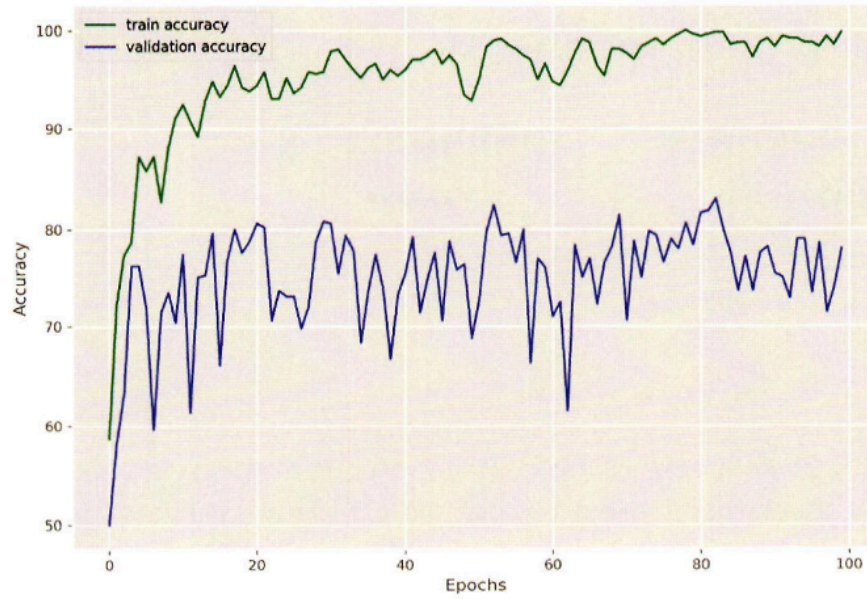
16it [04:39, 17.44s/it]

Validating

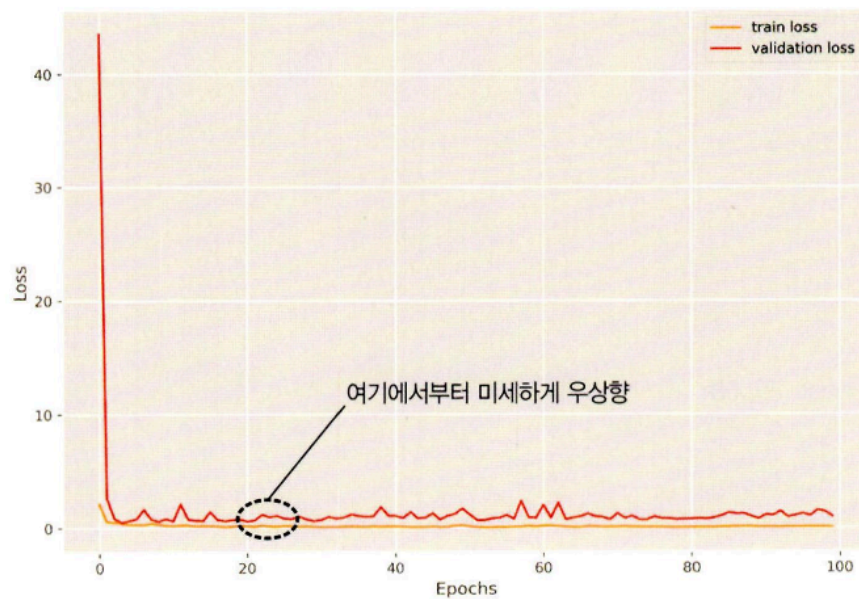
16it [01:30, 5.64s/it]

```
Train Loss: 0.4854, Train Acc: 76.51
Val Loss: 0.7689, Val Acc: 66.60
... 중간 생략 ...
Epoch 97 of 100
Training
16it [04:41, 17.58s/it]
Validating
16it [01:29, 5.62s/it]
Train Loss: 0.0410, Train Acc: 98.19
Val Loss: 0.8700, Val Acc: 75.20
Epoch 98 of 100
Training
16it [04:40, 17.52s/it]
Validating
16it [01:30, 5.68s/it]
Train Loss: 0.1077, Train Acc: 95.98
Val Loss: 0.9775, Val Acc: 80.60
Epoch 99 of 100
Training
16it [04:44, 17.81s/it]
Validating
16it [01:30, 5.66s/it]
Train Loss: 0.1609, Train Acc: 94.38
Val Loss: 1.9502, Val Acc: 68.40
Epoch 100 of 100
Training
16it [04:43, 17.72s/it]
Validating
16it [01:29, 5.62s/it]
Train Loss: 0.0995, Train Acc: 96.59
Val Loss: 1.5691, Val Acc: 72.00
Training time: 618.836 minutes
Saving loss and accuracy plots...
Saving model...
TRAINING COMPLETE
```

▼ 그림 8-54 어떤 인수도 적용되지 않았을 때의 정확도



▼ 그림 8-55 어떤 인수도 적용되지 않았을 때의 오차



- 정확도의 경우 위아래로 많은 변동이 있다. 정확도가 10% 이상 차이가 나는 일부 에포크 사이는 회복이 매우 심함. 오차에 대한 그래프도 마찬가지.

2. 학습률 감소에 대한 결과 확인

```
python es-python_82.py --lr-scheduler
```

```

Computation device: cpu

25,557,032 total parameters.
25,557,032 training parameters.
INFO: Initializing learning rate scheduler
Epoch 1 of 100
Training
  0%!
! 0/15 [00:00<?, ?it/s]C:\Users\Metanet\AppData\Roaming\Python\Python38\site-packages\
torch\nn\functional.py:718: UserWarning: Named tensors and all their associated APIs
are an experimental feature and subject to change. Please do not use them for anything
important until they are released as stable. (Triggered internally at ..\c10\core\
TensorImpl.h:1156.)
  return torch.max_pool2d(input, kernel_size, stride, padding, dilation, ceil_mode)
16it [09:02, 33.91s/it]
Validating
16it [02:08, 8.06s/it]
Train Loss: 2.1396, Train Acc: 61.04
Val Loss: 6.2706, Val Acc: 37.80
Epoch 2 of 100
Training
16it [05:41, 21.32s/it]
Validating
16it [01:16, 4.80s/it]
Train Loss: 0.5996, Train Acc: 69.68
Val Loss: 1.3892, Val Acc: 51.20
Epoch 3 of 100
Training
16it [04:17, 16.11s/it]
Validating
16it [01:15, 4.75s/it]
Train Loss: 0.5297, Train Acc: 74.50
Val Loss: 0.7101, Val Acc: 67.00
... 중간 생략 ...
Epoch 97 of 100
Training
16it [03:58, 14.91s/it]
Validating
16it [01:16, 4.78s/it]
Train Loss: 0.0017, Train Acc: 100.00
Val Loss: 0.5876, Val Acc: 83.40

```

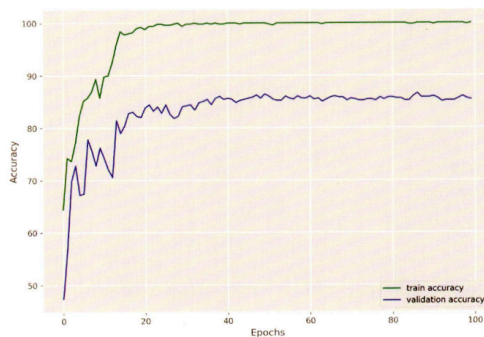


```

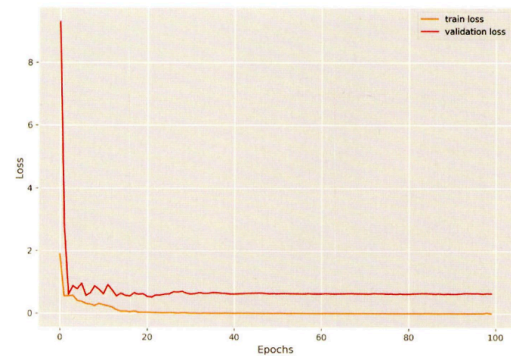
Epoch 98 of 100
Training
16it [04:02, 15.13s/it]
Validating
16it [01:16, 4.78s/it]
Train Loss: 0.0017, Train Acc: 100.00
Val Loss: 0.5880, Val Acc: 84.00
Epoch 99 of 100
Training
16it [04:01, 15.08s/it]
Validating
16it [01:14, 4.69s/it]
Train Loss: 0.0019, Train Acc: 100.00
Val Loss: 0.5919, Val Acc: 83.80
Epoch 100 of 100
Training
16it [03:59, 14.94s/it]
Validating
16it [01:14, 4.64s/it]
Train Loss: 0.0023, Train Acc: 100.00
Val Loss: 0.5959, Val Acc: 83.60
Training time: 589.638 minutes
Saving loss and accuracy plots...
Saving model...
TRAINING COMPLETE

```

▼ 그림 8-56 학습률 감소에 대한 인수가 적용되었을 때의 정확도



▼ 그림 8-57 학습률 감소에 대한 인수가 적용되었을 때의 오차



- 정확도 그래프가 완만한 곡선 형태. 훈련이 종료된 시점의 검증 데이터셋에 대한 정확도도 높음. 오차 그래프의 우상향 현상이 사라짐. 즉, 학습률 스케줄러가 성능 향상에 어느 정도 기여함.

3. 조기 종료 인수를 적용했을 때의 결과

```
python es-python_8%.py --early-stopping
```



```

Computation device: cpu

25,557,032 total parameters.
25,557,032 training parameters.
INFO: Initializing early stopping
Epoch 1 of 100
Training
  0%!
| 0/15 [00:00<?, ?it/s]e:\Anaconda3\envs\pytorch\lib\site-packages\torch\nn\
functional.py:718: UserWarning: Named tensors and all their associated APIs are an

```

```

experimental feature and subject to change. Please do not use them for anything
important until they are released as stable. (Triggered internally at
TensorImpl.h:1156.)

```

```

    return torch.max_pool2d(input, kernel_size, stride, padding, dilation, ceil_mode)

```

```

16it [04:45, 17.84s/it]

```

```

Validating

```

```

16it [01:34, 5.91s/it]

```

```

Train Loss: 2.1789, Train Acc: 58.23

```

```

Val Loss: 64.6766, Val Acc: 14.60

```

```

Epoch 2 of 100

```

```

Training

```

```

16it [04:31, 16.94s/it]

```

```

Validating

```

```

16it [01:29, 5.60s/it]

```

```

Train Loss: 0.6728, Train Acc: 65.66

```

```

Val Loss: 1.6799, Val Acc: 64.60

```

```

Epoch 3 of 100

```

```

Training

```

```

16it [04:33, 17.09s/it]

```

```

Validating

```

```

16it [01:31, 5.74s/it]

```

```

Train Loss: 0.5560, Train Acc: 70.28

```

```

Val Loss: 0.5631, Val Acc: 71.40

```

```

Epoch 4 of 100

```

```

Training

```

```

16it [04:46, 17.90s/it]

```

```

Validating

```

```

16it [01:34, 5.93s/it]

```

```

Train Loss: 0.5004, Train Acc: 79.92

```

```

Val Loss: 0.5125, Val Acc: 77.40

```

```

Epoch 5 of 100

```

```

Training

```

```

16it [04:47, 17.94s/it]

```

```

Validating

```

```

16it [01:34, 5.90s/it]

```

```

EarlyStopping counter: 1 out of 5

```

```

Train Loss: 0.3710, Train Acc: 83.53

```

```

Val Loss: 0.6830, Val Acc: 72.20

```

```

Epoch 6 of 100

```

```

Training

```

```

16it [04:46, 17.94s/it]

```

```

Validating

```

```

16it [01:34, 5.90s/it]

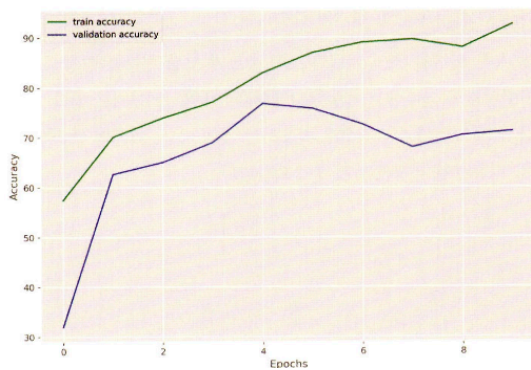
```

```

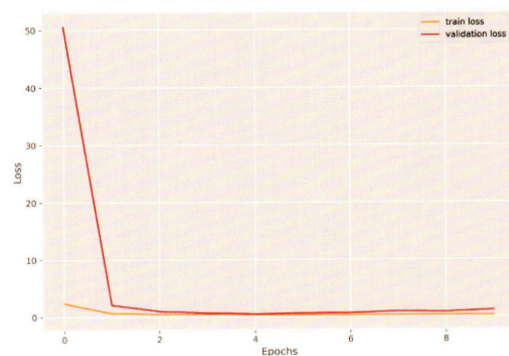
EarlyStopping counter: 2 out of 5
Train Loss: 0.4656, Train Acc: 78.11
Val Loss: 0.7071, Val Acc: 70.40
Epoch 7 of 100
Training
16it [04:36, 17.26s/it]
Validating
16it [01:30, 5.66s/it]
EarlyStopping counter: 3 out of 5
Train Loss: 0.4010, Train Acc: 83.33
Val Loss: 0.5996, Val Acc: 74.80
Epoch 8 of 100
Training
16it [04:34, 17.17s/it]
Validating
16it [01:30, 5.68s/it]
EarlyStopping counter: 4 out of 5
Train Loss: 0.3679, Train Acc: 83.94
Val Loss: 1.4061, Val Acc: 58.00
Epoch 9 of 100
Training
16it [04:34, 17.14s/it]
Validating
16it [01:30, 5.68s/it]
EarlyStopping counter: 5 out of 5
Training time: 55.807 minutes
Saving model...
TRAINING COMPLETE

```

▼ 그림 8-58 조기 종료에 대한 인수가 적용되었을 때의 정확도



▼ 그림 8-59 조기 종료에 대한 인수가 적용되었을 때의 오차



- 다섯 번째 에포크부터 조기 종료가 수행되고 있기 때문에 실제로 학습은 네번째 에포크까지만 수행됨. 조기 종료로 모델의 성능이 좋아지는 것이 아니기 때문에 모델이 제대로 학습하지 못할 수도 있음. 그렇다고 학습을 계속 이어간다고 해서 더 좋은 결과를 얻을 수 있을 거라는 보장도 없기 때문에 그래프가 의미하는 내용을 잘 이해하고 적절한 성능 향상 방안을 적용하는 것이 중요하다.